

Tool Calling in LangChain

This topic is one of the most important steps in understanding how **AI Agents** work.

In the previous lesson, you learned:

- What Tools are
- Built-in Tools
- Custom Tools
- Toolkits

Now the question is:

How does an LLM actually use a tool?

The answer is:

Tool Calling

1. Intro

Tools by themselves are just functions.

For example:

```
def multiply(a, b):  
    return a * b
```

The function exists.

But:

The LLM doesn't automatically know when to use it.

We need a mechanism that allows the model to:

1. Understand available tools
2. Decide which tool is needed
3. Generate tool inputs
4. Execute the tool
5. Use the result

This process is called:

Tool Calling

2. Quick Revision

Before Tool Calling:

LLM Only

```
User → LLM → Response
```

With Tools

```
User  
↓  
LLM  
↓  
Tool  
↓
```

LLM
↓
Response

The LLM becomes capable of interacting with external systems.

3. Tool Binding

What is Tool Binding?

Tool Binding means:

Attaching tools to an LLM so the model knows those tools are available.

Analogy

Imagine hiring a new employee.

You give them:

- Calculator
- Internet access
- Database access

Now they know what resources they can use.

Similarly:

```
llm_with_tools = llm.bind_tools(tools)
```

What Happens?

LangChain sends the tool information to the model.

The model learns:

- Tool name
- Tool description
- Inputs required

Example

Tool:

@tool

```
def multiply(a: int, b: int):  
    return a * b
```

Description:

Useful for multiplying two numbers.

After binding:

The model knows:

I can use a tool called multiply
when multiplication is needed.

Why Tool Descriptions Matter

The LLM chooses tools based on descriptions.

Bad Description

Does something.

The model gets confused.

Good Description

Useful for multiplying two integers.

The model clearly understands when to use it.

4. Tool Calling

What is Tool Calling?

Tool Calling is:

The process where the model decides a tool is needed and generates the tool request.

Important Point

At this stage:

Tool is NOT executed yet.

The model only says:

"I want to use this tool."

Example

User:

What is 25×17 ?

Model:

```
{
  "tool": "multiply",
  "args": {
    "a": 25,
    "b": 17
  }
}
```

This is a tool call.

The model has chosen:

- Tool = multiply
 - Inputs = 25, 17
-

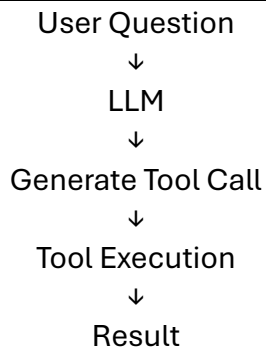
Key Idea

The model is not solving the problem.

The model is requesting:

"Please execute this tool for me."

Tool Calling Flow



5. Tool Execution

After tool calling comes:

Tool Execution

What Happens?

LangChain executes the tool.

Example:

```
Tool call:
{
  "tool": "multiply",
  "args": {
    "a": 25,
    "b": 17
  }
}
```

LangChain runs:

```
multiply(25,17)
```

Output:

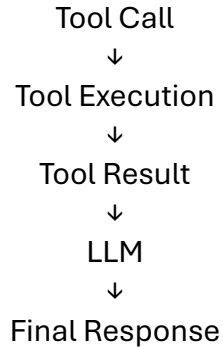
425

Then?

The result goes back to the LLM.

Flow:





Final Response

$25 \times 17 = 425$

Why Tool Execution Is Separate

This is extremely important.

LLM's Job

Decide:

Which tool?

What inputs?

LangChain's Job

Execute:

Run the tool

Get the result

Tool's Job

Perform:

Actual computation

Each component has a separate responsibility.

6. Currency Conversion Tool Example

This is a practical example of Tool Calling.

Problem

User asks:

Convert 100 USD to PKR

Step 1: Tool Binding

Bind currency tool.

```
llm.bind_tools([currency_tool])
```

Step 2: User Query

Convert 100 USD to PKR

Step 3: Tool Calling

Model generates:

```
{
  "tool": "currency_converter",
  "args": {
    "amount": 100,
    "from_currency": "USD",
    "to_currency": "PKR"
  }
}
```

Step 4: Tool Execution

Tool contacts exchange-rate API.

Returns:

28250 PKR

(Example value)

Step 5: Final Response

100 USD is approximately 28,250 PKR.

Why This Is Powerful

Without tools:

LLM guesses exchange rate

With tools:

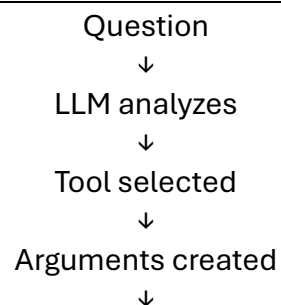
Uses live data

Behind the Scenes

What the user sees:

Convert 100 USD to PKR

What actually happens:



Tool executed
↓
Result returned
↓
Response generated

7. Is This an AI Agent?

This is one of the most important conceptual questions.

Short Answer

Not necessarily.

Tool Calling alone does NOT automatically make something an Agent.

What We Currently Have

User
↓
LLM
↓
Tool
↓
Response

This is:

Tool-Enabled LLM

or

Tool Calling System

What Is an AI Agent?

An Agent can:

1. Reason

What should I do?

2. Plan

Step 1

Step 2

Step 3

3. Choose Tools

Use Search

Use Calculator

Use Database

4. Execute Multiple Actions

Search
↓
Analyze
↓
Calculate
↓
Respond

5. Iterate

Need more information?
Use another tool.

Example

User:

Find the population of Pakistan and calculate
2% of it.

Agent:

1. Search population
2. Get result
3. Use calculator
4. Compute 2%
5. Return answer

Multiple reasoning steps occur.

Tool Calling vs Agent

Feature	Tool Calling	Agent
Uses Tools	✓	✓
One Tool Call	✓	✓
Multiple Tool Calls	Possible	✓
Planning	✗	✓
Reasoning	Limited	✓
Autonomous Decisions	✗	✓

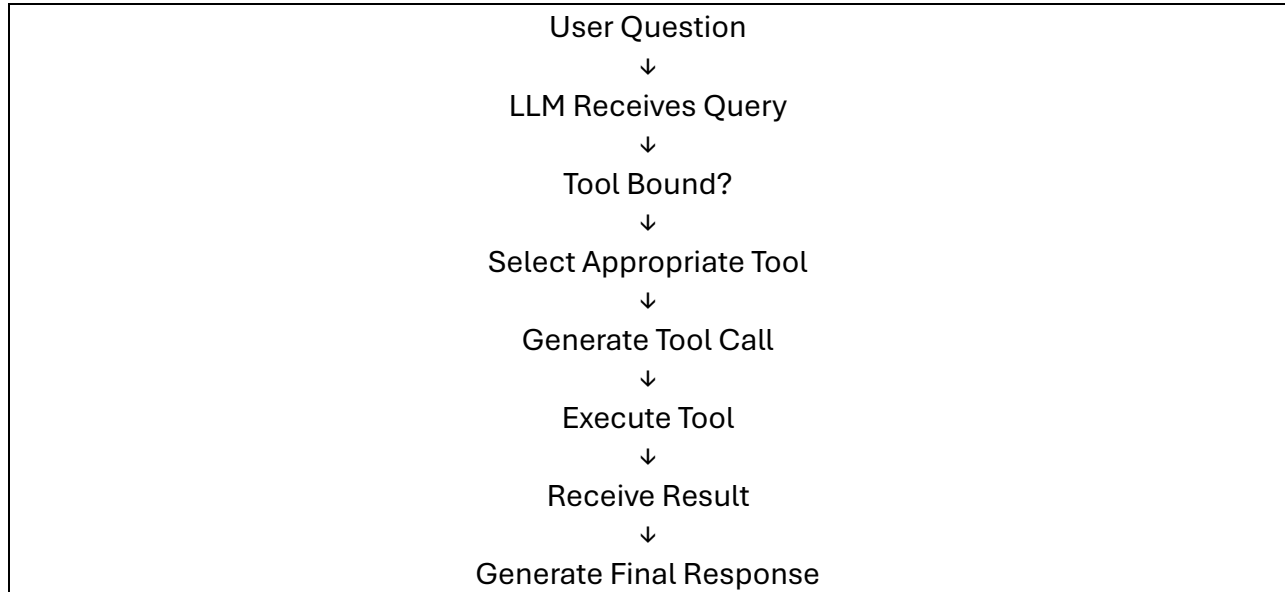
Relationship Between Them

Think of it like this:

Tool
↓
Tool Calling
↓
Agent

Tools are the foundation.
Tool Calling enables tool usage.
Agents build intelligent reasoning on top.

Complete Workflow



Big Picture Understanding

Before Tool Calling:
LLM = Text Generator

After Tool Calling:
LLM = Text Generator + External Abilities

After Agents:
LLM = Reasoning System + Tool User + Planner

Summary

Tool Binding

Makes tools available to the model.

Tool Calling

Model decides which tool to use and creates tool arguments.

Tool Execution

LangChain runs the tool and gets results.

Currency Conversion Example

Shows how live APIs can be integrated through tools.

Tool Calling ≠ Agent

Tool Calling is a capability.

An Agent is a system that can:

- Reason
- Plan
- Choose tools
- Execute multiple steps autonomously

Tool Calling is the mechanism that allows an LLM to decide when and how to use external tools, while agents are intelligent systems that use tool calling as part of a larger reasoning and planning process.